

Deployment

Reclaim runs as **one process**: the API serves the built dashboard from `web/dist`, so there is no separate frontend host, no CORS configuration, and no proxy to keep in sync.

```
Browser → Node process (:3000) → Neon Postgres
      |
      |— /          dashboard (web/dist)
      |— /api/*     JSON
```

What you need

Node	22 or newer
Postgres	Neon, or any Postgres 17
Razorpay	Optional — the simulator is the default
Python	Only to regenerate data or the evaluation report; not needed at runtime

The evaluation report is committed as `evaluation/report.json`, so the dashboard's Evaluation tab works on a deployed instance without Python present.

1 · Local (single service)

The closest thing to production, and what you should test before deploying.

```
npm install
cd web && npm install && npm run build && cd ..

# .env
DATABASE_URL=postgresql://...
PAYMENT_CLIENT=simulated

npm run api          # http://localhost:3000 – dashboard and API together
```

Verify:

```
curl -s localhost:3000/api/overview | head -c 80    # JSON
curl -s -o /dev/null -w '%{http_code}\n' localhost:3000/  # 200
```

If the startup line says *"dashboard not built"*, run the `web` build step above.

Development mode

For iterating on the dashboard, run the two separately — Vite gives hot reload and proxies

```
npm run api          # terminal 1 - :3000
npm run web          # terminal 2 - :5173, proxies /api
```

2 · Seeding a fresh database

Only needed for a brand-new Postgres. Neon is already seeded.

```
# schema
psql "$DATABASE_URL" -f src/db/migrations/001_initial_schema.sql

# data – deterministic from the seed, so anyone gets identical rows
python data/generator/generate.py --seed 20260831
python data/generator/load.py

# decisions and actions
npm run detect
npm run actions
npm run eval
```

Roughly 90 seconds end to end. Every step is idempotent: `npm run detect --reset` clears and rebuilds, and the generator's seed guarantees the same 5,000 attempts every time.

3 · Render (recommended)

Free tier, native Node support, no Docker needed.

1. Push to GitHub (already done).
2. **New** → **Web Service**, connect `Anamiiikka/Reclaim`.
3. Settings:

Field	Value
Runtime	Node
Build command	<code>npm install && cd web && npm install && npm run build</code>
Start command	<code>npx tsx src/api/server.ts</code>
Instance type	Free

1. Environment variables:

```
`` DATABASE_URL postgresql://... (from Neon, the pooled connection string) PAYMENT_CLIENT
simulated NODE_VERSION 22 ``
```

Add `RAZORPAY_KEY_ID` and `RAZORPAY_KEY_SECRET` only if you set `PAYMENT_CLIENT=razorpay`. **Never paste a live key** — the client refuses anything not starting with `rzp_test_`.

1. Deploy. First build takes ~3 minutes.

Free tier sleeps after 15 minutes idle and takes ~30 seconds to wake. If you put the URL in your submission, open it a minute before anyone looks at it, or note the cold start in the README.

4 • Railway

```
npm i -g @railway/cli
railway login
railway init
railway variables set DATABASE_URL="postgresql://..." PAYMENT_CLIENT=simulated
railway up
```

Railway reads `package.json`. Add this if it does not detect the build:

```
"scripts": {
  "build": "cd web && npm install && npm run build",
  "start": "tsx src/api/server.ts"
}
```

No sleep on the paid tier; the trial credit covers a demo comfortably.

5 • Fly.io

Needs a Dockerfile, but gives a region close to your reviewers.

```
FROM node:22-slim
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY web/package*.json ./web/
RUN cd web && npm ci
COPY . .
RUN cd web && npm run build
EXPOSE 3000
CMD ["npx", "tsx", "src/api/server.ts"]
```

```
fly launch --no-deploy
fly secrets set DATABASE_URL="postgresql://..." PAYMENT_CLIENT=simulated
fly deploy
```

6 · What not to do

Do not deploy to Vercel or Netlify as-is. Both are built for serverless functions with short execution limits and no persistent process. Reclaim's job worker (`npm run actions`) is a long-running loop holding database locks, and the API keeps a connection pool. It would appear to work and then fail in ways that are tedious to debug. Render, Railway and Fly all run a normal long-lived Node process, which is what this is.

Do not commit `.env`. It is gitignored, CI fails the build if a credential appears in a commit, and the scanner tests itself against a planted key on every run.

Do not set `PAYMENT_CLIENT=razorpay` on a public deployment unless you want strangers triggering calls against your test account's rate limit. The simulator is the honest default for a demo, and every payment link it creates is clearly marked `plink_sim_`.

Post-deploy checklist

```
curl -s https://YOUR-URL/api/overview | head -c 120
curl -s -o /dev/null -w '%{http_code}\n' https://YOUR-URL/
```

Then in a browser:

- [] Overview loads, policy-violations card reads **0**
- [] Evaluation tab renders (needs `evaluation/report.json` , which is committed)
- [] A deep link works: `https://YOUR-URL/#/case/rcv_00366`
- [] That case shows two attempts, no payment link, "no customer was contacted"

If the Evaluation tab is empty, `evaluation/report.json` did not deploy — check it is not caught by a `.gitignore` rule on the host.

Troubleshooting

Symptom	Cause	Fix
dashboard not built on startup	web/dist missing	cd web && npm run build
Dashboard loads, all data blank	DATABASE_URL unset or wrong	Check the env var; Neon needs ? sslmode=require
EADDRINUSE	Port already held	Kill the old process, or set PORT
Evaluation tab empty	report.json absent	npm run eval , or confirm it deployed
Slow first request	Neon compute suspended	Normal — it wakes in a few seconds
429s from Razorpay	Test-mode quota spent	Switch to PAYMENT_CLIENT=simulated

Is deploying worth it?

For the buildathon: **optional**. The required deliverables are a public repo, a five-minute video, architecture docs, a working demo and measured results. The video *is* the working demo, and screenshots are committed under docs/screenshots/ .

A live URL is a small bonus. Recording the video is not — it is a hard requirement. If time is short, record first and deploy afterwards.